

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from scipy import stats
```

```
from google.colab import files
uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving marketing_sales_data.csv to marketing_sales_data (2).csv

```
df = pd.read_csv('marketing_sales_data.csv')
df.head()
```

	TV	Radio	Social Media	Influencer	Sales
0	Low	3.518070	2.293790	Micro	55.261284
1	Low	7.756876	2.572287	Mega	67.574904
2	High	20.348988	1.227180	Micro	272.250108
3	Medium	20.108487	2.728374	Mega	195.102176
4	High	31.653200	7.776978	Nano	273.960377

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 572 entries, 0 to 571
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   TV               572 non-null   object  
1   Radio            572 non-null   float64 
2   Social Media     572 non-null   float64 
3   Influencer       572 non-null   object  
4   Sales            572 non-null   float64 
dtypes: float64(3), object(2)
memory usage: 22.5+ KB
```

```
df.isnull().sum()
```

	0
TV	0
Radio	0
Social Media	0
Influencer	0
Sales	0

dtype: int64

```
df = df.dropna()  
df.isnull().sum()
```

	0
TV	0
Radio	0
Social Media	0
Influencer	0
Sales	0

dtype: int64

```
df.describe()
```

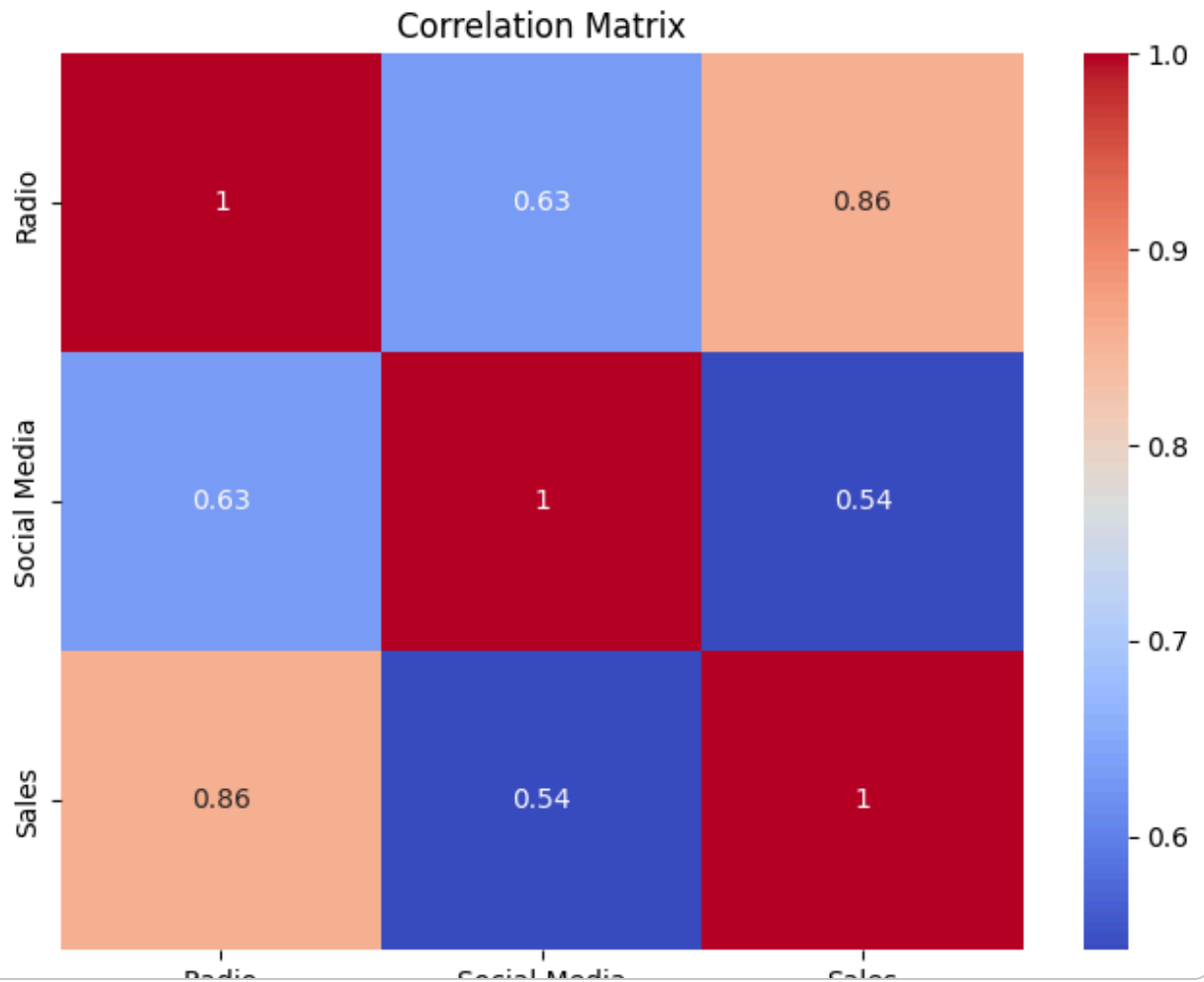
	Radio	Social Media	Sales
count	572.000000	572.000000	572.000000
mean	17.520616	3.333803	189.296908
std	9.290933	2.238378	89.871581
min	0.109106	0.000031	33.509810
25%	10.699556	1.585549	118.718722
50%	17.149517	3.150111	184.005362
75%	24.606396	4.730408	264.500118
max	42.271579	11.403625	357.788195

```

numeric_df = df.select_dtypes(include=np.number)
corr = numeric_df.corr()

plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()

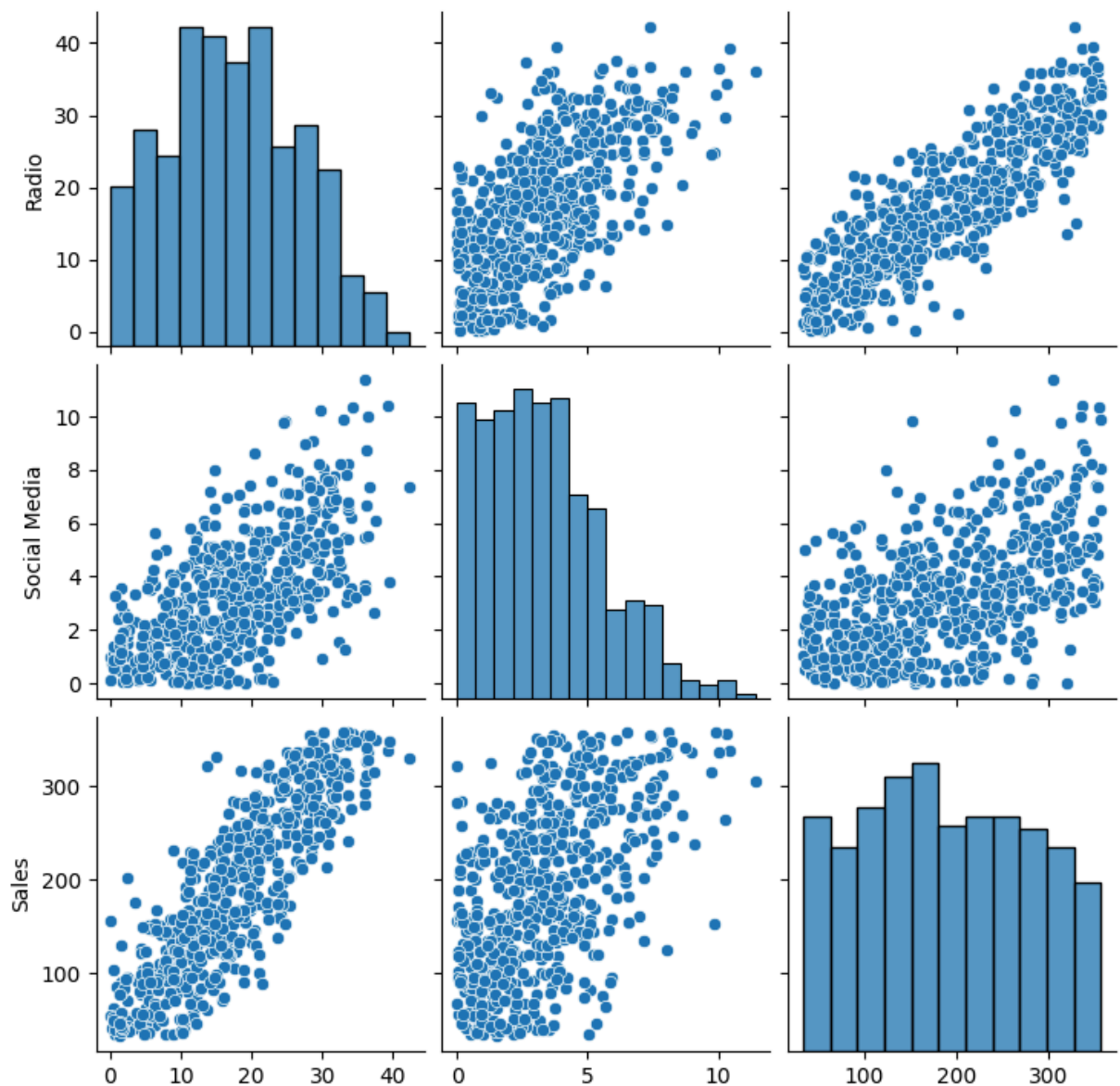
```



```

sns.pairplot(df)
plt.show()

```



```
X = df[['Radio', 'Social Media']]

vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns

vif_data["VIF"] = [
    variance_inflation_factor(X.values, i)
    for i in range(len(X.columns))
]

print(vif_data)
```

	Feature	VIF
0	Radio	5.170922
1	Social Media	5.170922

Multicollinearity Check

Variance Inflation Factor (VIF) values below 5 indicate that multicollinearity is not a serious concern.

```
X = df[['TV', 'Radio', 'Social Media']]
y = df['Sales']

# Convert categorical 'TV' column to numerical using one-hot encoding
X = pd.get_dummies(X, columns=['TV'], drop_first=True)

# Convert boolean columns created by get_dummies to integers
X = X.astype(int)

X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print(model.summary())
```

OLS Regression Results					
=====					
Dep. Variable:	Sales	R-squared:			
Model:	OLS	Adj. R-squared:			
Method:	Least Squares	F-statistic:			
Date:	Mon, 08 Jun 2026	Prob (F-statistic):		3.08	
Time:	12:38:51	Log-Likelihood:		-2	
No. Observations:	572	AIC:			
Df Residuals:	567	BIC:			
Df Model:	4				
Covariance Type:	nonrobust				
=====					
	coef	std err	t	P> t	[0.025

const	219.7180	6.167	35.630	0.000	207.606
Radio	3.0098	0.233	12.894	0.000	2.551
Social Media	-0.2213	0.672	-0.329	0.742	-1.541
TV_Low	-153.9883	4.931	-31.228	0.000	-163.674
TV_Medium	-75.1405	3.626	-20.724	0.000	-82.262
=====					
Omnibus:	60.195	Durbin-Watson:			
Prob(Omnibus):	0.000	Jarque-Bera (JB):		1	
Skew:	0.046	Prob(JB):		0.0	
Kurtosis:	2.138	Cond. No.			
=====					
Notes:					
[1] Standard Errors assume that the covariance matrix of the errors is co					

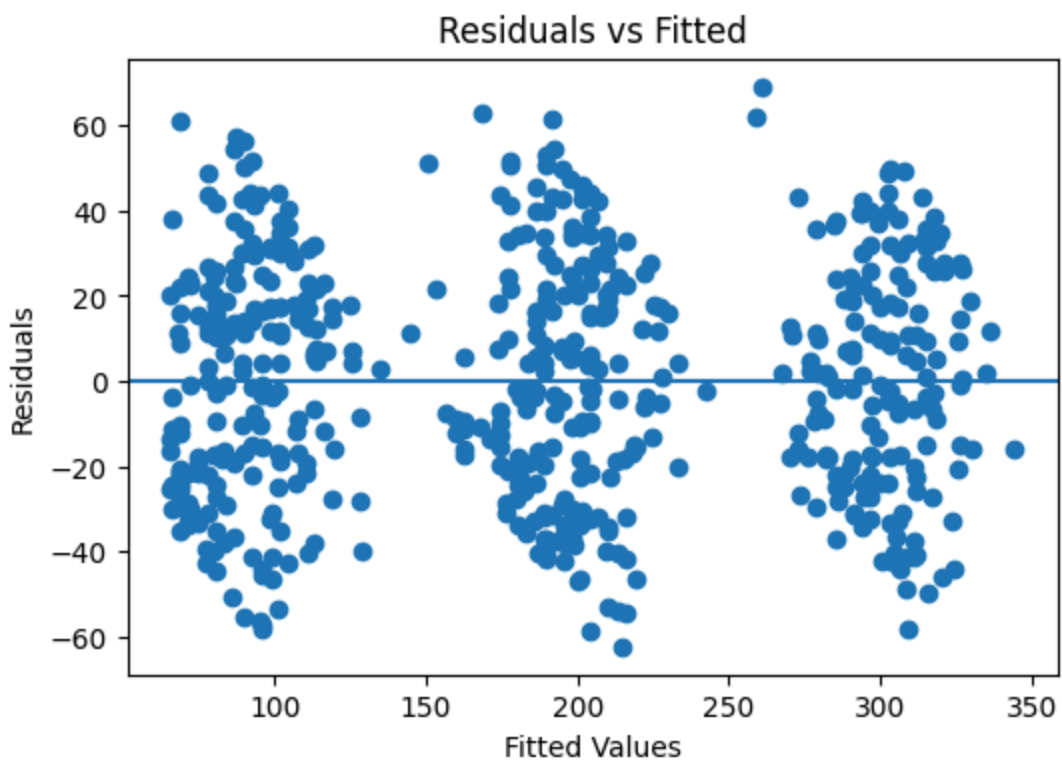
Model Evaluation

The model is evaluated using:

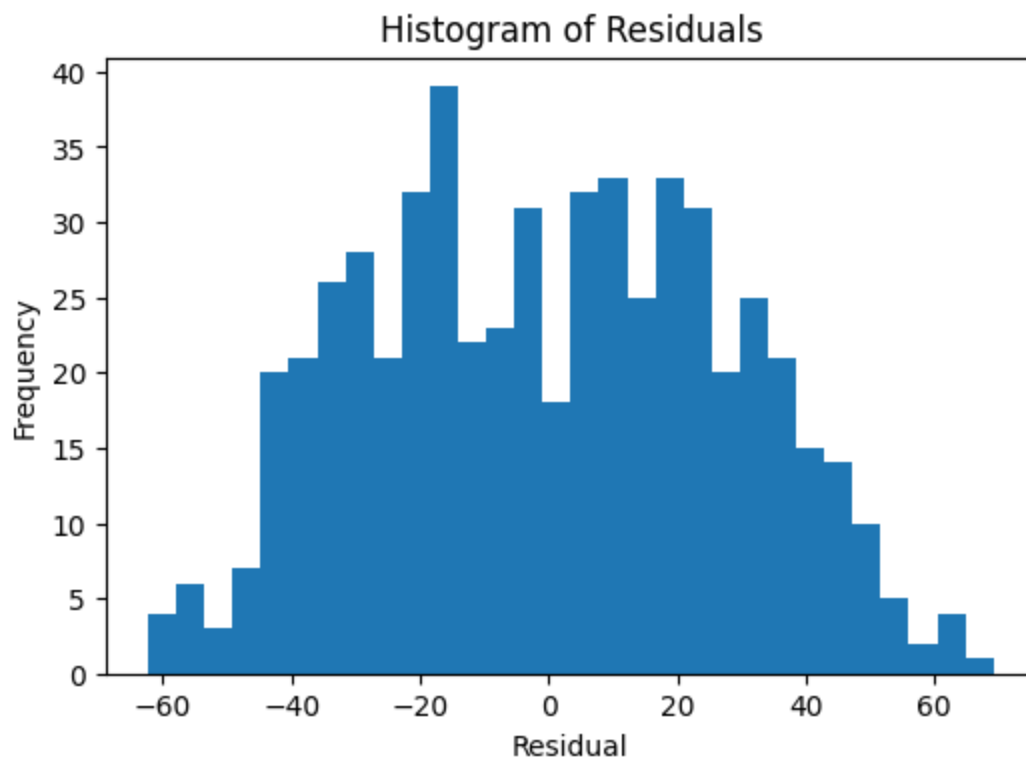
- Adjusted R-squared
- Predictor p-values
- Coefficient estimates

```
predictions = model.predict(X)
residuals = y - predictions
```

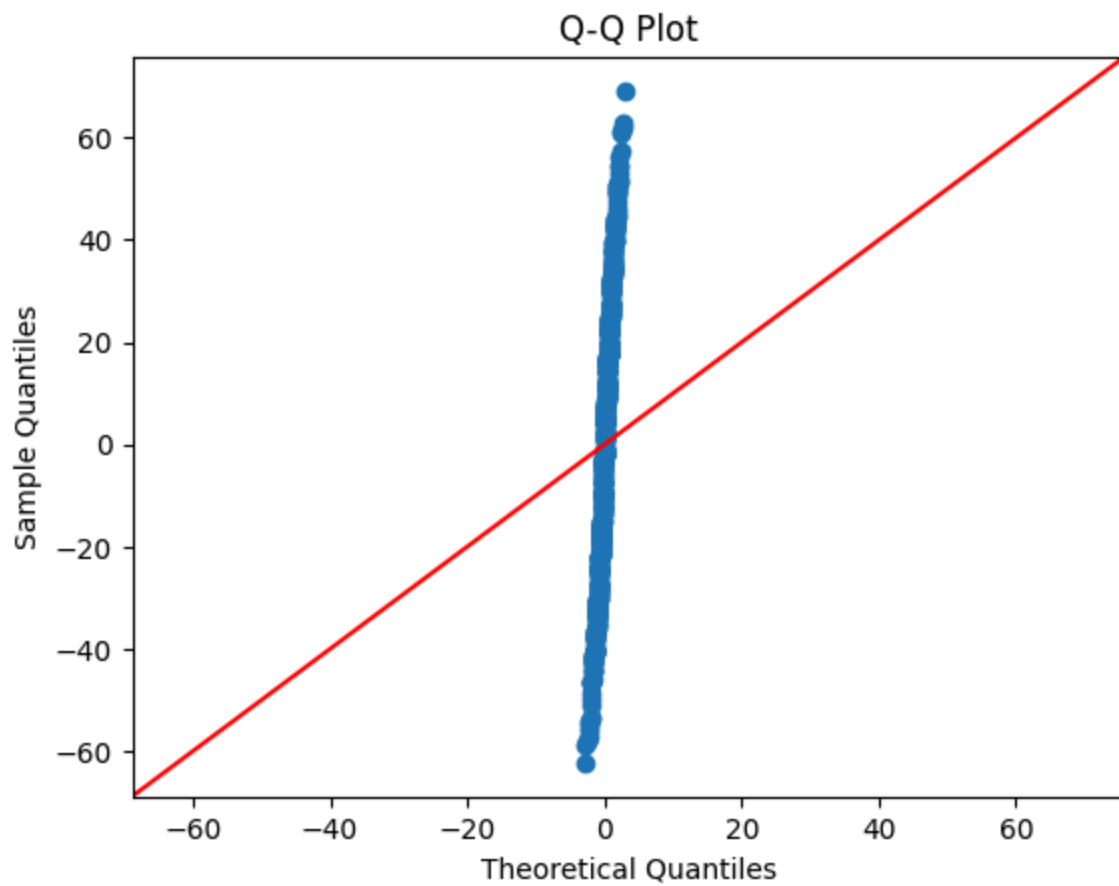
```
plt.figure(figsize=(6,4))
plt.scatter(predictions, residuals)
plt.axhline(y=0)
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted')
plt.show()
```



```
plt.figure(figsize=(6,4))
plt.hist(residuals, bins=30)
plt.title('Histogram of Residuals')
plt.xlabel('Residual')
plt.ylabel('Frequency')
plt.show()
```



```
sm.qqplot(residuals, line='45')  
plt.title('Q-Q Plot')  
plt.show()
```



Coefficient Interpretation

Holding Radio and Social Media advertising constant, a one-unit increase in TV advertising is associated with an increase in Sales equal to the estimated TV coefficient.

The same interpretation applies to Radio and Social Media coefficients.

Conclusion

Multiple Linear Regression was used to evaluate the combined impact of TV, Radio, and Social Media advertising on Sales.

Adjusted R-squared was used to assess overall model performance.